

ACURIA PARTNERS

Multi-Tenant Wealth Management Platform

Architecture Proposal — Internal Draft v1.0
Prepared by Altaaf Hamod · Solal Digital · June 2026

1. Context & Goal

This document outlines the proposed technical architecture for rebuilding the NM Prime wealth management dashboard as a scalable, multi-tenant SaaS platform. The objective is to allow Acuria Partners to onboard new wealth management firms (CGP/MFO) as independent tenants without duplicating code, data, or infrastructure.

The current dashboard is a single-tenant HTML/JS application deployed on Netlify with API secrets exposed in the source code. Phase 1 (security hardening) addresses the immediate production need. This proposal covers Phase 2: the full multi-tenant rebuild.

2. Proposed Technology Stack

Layer	Choice
Frontend	React (Vite) — single Vercel deployment
Backend	Express.js + Node.js — single Vercel deployment
Database	Baserow Cloud (Hetzner, Frankfurt — EU/GDPR compliant)
Authentication	JWT (access token + refresh token, managed by backend)
Automations	Make.com (EU data centre, tenant-aware scenarios)
E-signature	DocuSign (existing, kept)
Documents	SharePoint / Microsoft 365 (existing, kept)
AI Generation	Anthropic Claude API (Zero Data Retention addendum)
Hosting	Vercel (Frankfurt region for EU data residency)

Why Baserow?

- Client requirement: Baserow is the agreed-upon database layer.
- Non-technical admins can browse and audit data via the Baserow GUI without developer access.
- Hosted on Hetzner in Frankfurt — fully GDPR compliant, no CLOUD Act exposure.
- Workspace-level isolation provides strong tenant data separation enforced by Baserow itself, not just by our code.

3. Database Architecture

Baserow does not support PostgreSQL schemas or raw SQL. Tenant isolation is instead achieved through Baserow Workspaces — one workspace per tenant. Each workspace is fully siloed at the API level: a token scoped to Workspace A cannot read or write Workspace B, regardless of application-level logic.

3.1 Auth Workspace (_acuria_auth)

A single dedicated Baserow workspace holds all authentication and tenant configuration data. It is accessed exclusively by the backend using a master API token stored as a Vercel environment variable — never exposed to the browser.

Table	Fields
users	id, email, password_hash (bcrypt), tenant_id, role, created_at
tenants	id, name, slug, baserow_workspace_id, baserow_api_token, created_at

3.2 Per-Tenant Workspaces

Each tenant (e.g. NM Prime, DemoFinance) gets its own Baserow workspace with its own scoped API token. The backend stores this token in the tenants table and uses it for all business data queries for that tenant. Tables are provisioned automatically from a backend template on tenant creation.

Table	Purpose
clients	Client CRM records, KYC status, notes
documents	Generated regulatory documents (DER, LdM, FCC)
kyc_forms	FCC questionnaire submissions
tasks	Compliance to-do items per client
audit_log	Action history for RGPD/ACPR auditability

4. Authentication Flow

Login sequence

1. User submits email + password to POST /auth/login
2. Backend queries `_acuria_auth` workspace (master token) — finds user record
3. Verifies bcrypt password hash
4. Fetches tenant record — retrieves `workspace_id` and `api_token` for that tenant
5. Signs JWT: { `user_id`, `tenant_id`, `role` } — short-lived (15 min) + refresh token
6. Tenant API token is cached in backend memory — never sent to the browser

Every subsequent request

1. Frontend sends JWT in Authorization header
2. Backend middleware verifies JWT — extracts `tenant_id`
3. Resolves tenant Baserow token from memory cache (or re-fetches if expired)
4. All Baserow API calls use the tenant-scoped token — data isolation guaranteed

5. Automated Tenant Onboarding

The super admin triggers tenant creation from the platform UI. The backend handles the full provisioning sequence automatically via the Baserow API — no manual steps required.

Step	Action
Step 1	Super admin sends POST /admin/tenants { name, slug, admin_email }
Step 2	Backend calls Baserow API — creates new workspace named after the tenant
Step 3	Backend calls Baserow API — generates a scoped API token for that workspace
Step 4	Backend runs table provisioning template — creates all required tables in the new workspace
Step 5	Backend inserts tenant record into <code>_acuria_auth</code> (<code>workspace_id</code> + <code>api_token</code> stored)
Step 6	Backend creates the tenant admin user record in the users table
Step 7	Tenant admin receives an invite email to set their password

Result

A new tenant is fully operational in under 60 seconds from a single button click.

No code changes, no manual Baserow setup, no redeployment required.

6. Security Considerations

Concern	Mitigation
Tenant data isolation	Enforced by Baserow at API level — workspace-scoped tokens cannot cross tenant boundaries
Secret management	All API tokens stored as Vercel environment variables — never in source code or browser
Password storage	bcrypt hashing with salt — no plaintext passwords stored
JWT security	Short-lived access tokens (15 min) + HttpOnly refresh token cookies
CORS	Strict origin whitelist on Express backend
RGPD / ACPR	All data hosted in Frankfurt (EU) — DPA available from Baserow (Hetzner)
Audit trail	audit_log table per tenant — all sensitive actions recorded with timestamp + user
Super admin isolation	Master Baserow token only used for auth workspace — cannot access tenant workspaces

7. User Roles

Role	Scope & Permissions
Super Admin	Acuria internal — manages tenants, creates/removes tenant accounts, accesses platform-wide config
Tenant Admin	Per-tenant — manages users within their tenant, configures branding and settings
Standard User	Per-tenant — uses the dashboard (CRM, document generation, KYC) within their tenant only

8. Open Points for Team Review

The following items require a decision before development begins:

- Baserow plan: the free tier may have API rate limits that become an issue under load. Confirm whether the project budget covers a paid Baserow plan.
- Token refresh strategy: should the tenant Baserow API token be cached in-process memory (fast, lost on cold start) or stored in a lightweight KV store on Vercel? Cold starts on Vercel serverless functions mean in-memory cache is not persistent.
- Make.com scenario refactoring: existing workflows are mono-tenant. They need to be updated to accept a tenant identifier on each trigger. Confirm if this is in scope for Phase 2.
- Email provider: tenant admin invite emails and password reset flows require a transactional email service (e.g. Resend, which is already used on other projects).
- Vercel cold starts: Vercel serverless functions have cold start latency. If response time is critical, consider Vercel Pro or an always-on backend alternative (e.g. Railway, Render).

9. Summary

Architecture in one paragraph

A React frontend and an Express backend are each deployed to Vercel under a single URL. Baserow Cloud (Frankfurt) serves as the sole database. One dedicated Baserow workspace (`_acuria_auth`) holds user accounts and tenant configuration, accessed by the backend via a master API token stored in Vercel environment variables. Each tenant gets their own isolated Baserow workspace provisioned automatically on onboarding, with their own scoped API token that the backend resolves from the tenant record on every authenticated request. JWT-based authentication with short-lived tokens and bcrypt password hashing ensures secure access. All data remains in the EU (Hetzner, Frankfurt) for RGPD and ACPR compliance.